

TP1 — Primeira API REST

Ano letivo 2026/27 · 3.º ano · **Peso: 2%** · Trabalho individual · Entrega até **5 de outubro de 2026**

1. Objetivo

Consolidar, numa primeira entrega formal, o trabalho das Semanas 1 e 2: ambiente local, API REST, recursos, operações HTTP, CRUD em memória, validação, erros, testes e Git.

Neste TP1 não é exigida interface gráfica. A demonstração é feita através da API, com Postman, Insomnia ou ferramenta equivalente.

2. Regras gerais

- O trabalho é **individual**. Cada estudante tem o seu repositório e a sua entrega.
- O trabalho parte do projeto desenvolvido nas aulas S1 e S2.
- **IA generativa é permitida com declaração** (secção 8). A responsabilidade pela solução é sempre do estudante.
- Não se colocam em serviços externos, nem no repositório, palavras-passe, tokens, chaves ou dados pessoais.
- Os dados permanecem **em memória**: não se utiliza PostgreSQL nem ORM.

3. Ponto de partida e estrutura do projeto

O projeto executa localmente com Node.js (versão LTS; **indicar a versão usada no README**). A partir da pasta backend/:

- `npm install` instala as dependências;
- `node server.js` arranca a API (por defeito em `http://localhost:3000`).

Estrutura mínima esperada do repositório:

```
lei022-tp1-nome-apelido/  
├── backend/  
│   ├── package.json  
│   ├── package-lock.json  
│   └── server.js  
├── tests/           (coleção Postman/Insomnia exportada)  
├── README.md  
└── .gitignore       (node_modules/, .env, ...)
```

Outros ficheiros são permitidos quando necessários. **Não se versiona** `node_modules`, palavras-passe, tokens, chaves ou outros dados sensíveis.

4. Recurso obrigatório — items

Cada item tem `id` (número inteiro positivo, gerado pelo servidor) e `name` (texto não vazio; guarda-se sem espaços exteriores).

Método	Endpoint	Resultado
GET	/api/items	Consultar a coleção
GET	/api/items?name=...	Consultar a coleção com filtro
GET	/api/items/:id	Consultar um recurso
POST	/api/items	Criar recurso
PUT	/api/items/:id	Atualizar recurso
DELETE	/api/items/:id	Eliminar recurso

5. Regras mínimas

Situação	Resposta
Consulta ou atualização com sucesso	200 , com o recurso ou a coleção
Criação com sucesso	201 , com o recurso criado e o header Location: /api/items/<id>
Remoção com sucesso	204 , sem corpo
id que não é inteiro positivo (abc, 0, -1, 1e0)	400 , em GET, PUT e DELETE
Corpo inválido ou em falta; JSON malformatado	400 , nunca 500
Recurso ou rota inexistente	404
Filtro name sem resultados	200 e []

- O POST recebe os dados em JSON no corpo e **gera o id no servidor**. O cliente não escolhe o id.
- A pesquisa por name usa **query parameter**, ignora maiúsculas/minúsculas e espaços exteriores, e devolve os items cujo nome **contém** o termo.
- **Ordem das validações:** primeiro o que se verifica sem consultar os dados (id e corpo → 400); depois o estado (recurso inexistente → 404; conflito → 409).

Formato único de erro

Todas as respostas de erro (incluindo rota desconhecida, JSON malformatado e erros inesperados) usam o mesmo formato, inspirado no RFC 9457 (Problem Details). Não se expõem stack traces nem detalhes técnicos.

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{ "status": 404, "title": "Não encontrado", "detail": "Não existe item com id 42" }
```

6. Recurso adicional — à sua escolha

Aplique o mesmo padrão REST a um **segundo recurso, de um domínio à sua escolha** (por exemplo equipamentos, livros, salas, eventos), mantendo os dados em memória. Indique o domínio e os campos no README.

- Pelo menos **4 campos**: id e mais três.
- Um campo com **valores permitidos** (por exemplo estado: disponível, emprestado, avariado). Um valor fora do conjunto produz **400**.
- Um campo **gerado pelo servidor** (por exemplo criadoEm, em formato ISO 8601). O cliente não o pode definir e mantém-se num PUT.
- Uma regra de **conflito** (por exemplo, dois registos não podem ter o mesmo nome, ignorando maiúsculas). Violá-la produz **409**.
- Os cinco endpoints CRUD (GET coleção, GET por id, POST, PUT, DELETE) com as regras da secção 5.

7. Testes e evidências

Execute os testes abaixo para /api/items e, com os casos equivalentes, para o recurso adicional.

Teste	Esperado	Objetivo
GET /api/items	200	coleção
GET /api/items/1	200	recurso existente
GET /api/items/999	404	recurso inexistente
GET /api/items/abc	400	id inválido
GET /api/items?name=... (com maiúsculas e espaços)	200	filtro
GET /api/items?name=xyz	200 + []	sem resultados
POST válido	201 + Location	criação
POST inválido (ex.: {} ou name: 123)	400	validação
POST sem corpo JSON	400	corpo em falta
POST com JSON malformatado	400	corpo ilegível
PUT válido	200	atualização
PUT inexistente	404	recurso inexistente
PUT inválido	400	validação
PUT /api/items/abc	400	id inválido
DELETE válido	204	remoção
DELETE inexistente	404	recurso inexistente
DELETE /api/items/abc	400	id inválido
GET /api/xpto (rota desconhecida)	404	mesmo formato de erro
Recurso adicional: sucesso e erro	200/201/204/400/404	consolidação
Recurso adicional: valor fora do conjunto	400	valores permitidos
Recurso adicional: nome duplicado	409	conflito

Como entregar a evidência

- Guarde em tests/ a **coleção exportada** do Postman/Insomnia, com verificações do estado HTTP (por exemplo, `pm.test` no Postman).
- No README, inclua uma tabela **pedido · esperado · obtido** com os resultados da última execução.
- Testes automatizados são valorizados mas não obrigatórios.

8. Repositório, README e declaração de IA

- Repositório GitHub **privado e próprio**, sugestão de nome `lei022-tp1-nome-apellido`.
- Adicione os dois docentes como colaboradores, com as contas `fmcm67` e `ZephyrDogma`.
- O histórico Git deve mostrar a evolução realizada nas S1 e S2. O número de commits, por si só, não determina a classificação.

README

O README deve conter: identificação do estudante, versão de Node usada, **instalação, execução**, o domínio do recurso adicional, **como foram feitos os testes** (tabela da secção 7) e a declaração de IA. Deve incluir uma **tabela de endpoints** com as colunas:

Método	Endpoint	Parâmetros / corpo	Sucesso	Erros
GET	/api/items/:id	id (inteiro positivo)	200	400, 404

Declaração de utilização de IA

Se usou IA generativa, inclua no README uma secção “Utilização de IA” com: **ferramenta, finalidade, parte do projeto, alterações relevantes** que fez ao resultado e **como validou** (testes, leitura do código). Se não usou, escreva “Não foi utilizada IA”. Não é necessário entregar o histórico das conversas.

9. Vídeo de demonstração

Vídeo de **máximo 3 minutos**, sem edição, com a sua voz. É evidência complementar; não substitui o código nem os testes. Roteiro sugerido:

Tempo	O que mostrar
0:00 – 0:20	Arranque local da API (<code>node server.js</code>) e o repositório GitHub com o histórico.
0:20 – 1:50	Postman/Insomnia sobre <code>/api/items</code> : GET coleção, GET por id, pesquisa por name, POST, PUT e DELETE; pelo menos um caso de erro (400 e 404).
1:50 – 2:20	O recurso adicional: um caso de sucesso e um de erro (400 ou 409).
2:20 – 3:00	Explique uma decisão de implementação (por exemplo, como valida o id ou o corpo) e como a testou.

Coloque o vídeo numa pasta partilhada com acesso aos docentes e indique a ligação no Moodle.

10. Entrega no Moodle

Prazo: 5 de outubro de 2026. O Moodle regista a data e a hora. A submissão inclui:

- URL do repositório GitHub privado;
- **SHA do commit** a avaliar (`git rev-parse HEAD`);

- ligação do vídeo.

Antes de submeter: `git status` limpo, `git push` feito e confirmação no GitHub de que o commit indicado **está publicado**. Opcionalmente, marque a entrega com `git tag tp1-entrega` e `git push --tags`.

A avaliação incide sobre o commit indicado. Alterações posteriores não o substituem, salvo indicação dos docentes. Os docentes podem clonar o repositório, fazer checkout desse commit, instalar as dependências e executar a API.

Política de entregas tardias: [não permitido]

Verificação oral: [opcional e escolha aleatória de alunos — 2–3 minutos no início da PL da S3: executar a API e explicar um endpoint]

11. Avaliação (100 pontos → 2% da nota final)

Critério	Pontos	O que se avalia
Arranque e reprodução	10	A API arranca seguindo o README (<code>npm install + node server.js</code>); working tree limpo.
Endpoints de items e códigos HTTP	25	Os seis endpoints; 200, 201 (com Location) e 204 corretos; filtro name conforme as regras.
Validação e erros	15	400, 404 nos casos previstos; formato único de erro; nenhum pedido dá 500.
Recurso adicional	20	CRUD completo, valores permitidos, campo gerado pelo servidor, 409.
Testes	10	Coleção exportada e tabela pedido/esperado/obtido coerente com o comportamento real.
Git, README e IA	10	Histórico com evolução; README com tabela de endpoints; declaração de IA.
Vídeo e explicação	10	Cumprimento do roteiro e explicação corretamente de uma decisão.

12. Checklist antes da entrega

- ☐ A API arranca a partir de backend/ com `npm install` e `node server.js`.
- ☐ items tem GET coleção, GET por id, POST (201 com Location), PUT e DELETE.
- ☐ A pesquisa por name ignora maiúsculas e espaços exteriores.
- ☐ Há 400 e 404 nos casos previstos, incluindo id inválido em PUT e DELETE; nenhum pedido inválido dá 500.
- ☐ Todos os erros têm o mesmo formato (status, title, detail).
- ☐ O recurso adicional tem CRUD completo, valores permitidos, campo gerado pelo servidor e 409.
- ☐ A coleção de testes está em tests/ e o README tem a tabela de resultados e a tabela de endpoints.
- ☐ O README tem instalação, execução, testes e declaração de IA.
- ☐ `git status` limpo, `git push` feito e o commit está visível no GitHub.
- ☐ O repositório é privado e os dois docentes têm acesso.
- ☐ O vídeo (máx. 3 min) foi submetido e a ligação funciona.
- ☐ O SHA indicado no Moodle corresponde à versão a avaliar.

Nota: este enunciado concretiza o TP1 a partir das atividades das S1 e S2. A interface Web não é objeto desta entrega; a integração com o frontend será trabalhada posteriormente.